The λ_S Hierarchy:
A Stratified Formal Calculus for Session-Typed Objects

Alexandre Zua Caldeira
Independent Researcher
azuacaldeira@gmail.com

March 2026

Abstract

We present the λ_S hierarchy: five calculi of increasing expressiveness that formalise the session-type discipline for objects. λ_S^0 captures straight-line typestate checking with Unit returns. λ_S^1 adds base-type returns and proves they are orthogonal to the session discipline. λ_S^2 adds selection types and enum returns, breaking this orthogonality: return values now determine protocol branches. λ_S^3 adds recursive session types and loops, introducing cyclic state spaces and non-termination. λ_S^4 adds parallel composition ($S_1 \parallel S_2$), whose product state-space construction forces lattice structure—the central result connecting the hierarchy to the Reticulate Theorem. Each calculus is a conservative extension of its predecessor; all satisfy preservation and progress. We validate the hierarchy against BICA Reborn, a Java session-type checker, with 102 alignment tests establishing a precise correspondence between formal rules and implementation.

1 Introduction

Session types for objects [5, 8] specify the permitted sequences of method calls on a stateful object. Implementations such as BICA Reborn and Mungo typecheck session-type conformance, but the formal foundations connecting these implementations to a calculus have not been fully developed.

This paper presents the λ_S *hierarchy*: a family of five calculi that stratify session-typed object programming by feature complexity. The hierarchy is designed so that:

1. Each level adds exactly one feature to the session type grammar and one construct to the term language.
2. Each level is a *conservative extension* of the previous.
3. Each level has a precise *implementation correspondence* with the BICA Reborn typestate checker.
4. The top level (λ_S^4) connects to the Reticulate Theorem: the $\parallel$ constructor forces lattice structure [2].

The hierarchy (Table 1) reveals three phase transitions:

(a) $\lambda_S^1 \to \lambda_S^2$: orthogonality breaks. Return values start affecting protocol evolution.

(b) $\lambda_S^2 \to \lambda_S^3$: termination breaks. Recursive types create cyclic state spaces; programs may diverge.

(c) $\lambda_S^3 \to \lambda_S^4$: lattice structure emerges. The product construction forces meets and joins to exist.

Table 1: The λ_S hierarchy at a glance.

	Session types	Term addition	Terminates?	Key result	Rules
λ_S^0	&, **end**	let, call	Yes	Straight-line typestate	5
λ_S^1	&, **end**	+const	Yes	Orthogonality of returns	6
λ_S^2	&, $\oplus$, **end**	+match	Yes	Enum returns $\to$ branches	8
λ_S^3	$+\mu$	+while	No	Cyclic state spaces	10
λ_S^4	$+\|$	+fork/join	No	Product lattice	12

Contributions.

- Five calculi with formal syntax, typing rules, operational semantics, and soundness proofs (Sections 3 to 7).
- The Orthogonality Theorem for λ_S^1: base-type returns never affect the session-typed environment (Theorem 4.1).
- Thread safety for λ_S^4: concurrent fork branches access disjoint methods (Theorem 7.2).
- 102 alignment tests against BICA Reborn's implementation (Section 9).
- Lean 4 mechanisation of all five calculi (20 files).

Running example. A file handle with protocol:

$$S_{\mathrm{FH}} = \&\{\mathit{open} : (\&\{\mathit{read} : \mathbf{end}\} \parallel \&\{\mathit{write} : \mathbf{end}\}) \,.\, \&\{\mathit{close} : \mathbf{end}\}\}$$

We show this protocol at each level of the hierarchy, progressively adding return types (λ_S^1), selection (λ_S^2), recursion (λ_S^3), and concurrency (λ_S^4).

2 Common Infrastructure

All five calculi share:

Class table. $\mathcal{CT}$ maps class names C to state spaces $\mathcal{L}(S_C)$ with top $\top$ (initial), bottom $\bot$ (terminated), and partial transition function $\delta(s, m) = s'$.

Typing judgement. $\mathcal{CT};\ \Gamma \vdash e : T \dashv \Gamma'$, where Γ tracks session-typed variables $x : C \mathbin{@} s$.

Configurations. $\langle H,\ \sigma,\ e\rangle$ with heap H (location $\mapsto$ class), session store σ (location $\mapsto$ state), runtime expression e.

Soundness. *Preservation*: well-typed configurations reduce to well-typed configurations. *Progress*: well-typed non-values can reduce. *Session completion*: terminated programs have all locations at $\bot$.

3 λ_S^0: Straight-Line Typestate

Session types. $S ::= \&\{m_1 : S_1, \ldots\} \mid \mathbf{end}$. State spaces are finite trees.

Syntax. $e ::= v \mid \mathbf{new}\ C() \mid x.m() \mid \mathbf{let}\ x = e_1\ \mathbf{in}\ e_2, \quad v ::= x \mid \mathbf{skip}$.

Key rule.

$$\text{T-Call}^0 \ \frac{\Gamma(x) = C \mathbin{@} s \qquad \delta(s, m) = s'}{\mathcal{CT};\ \Gamma \vdash x.m() : \mathbf{Unit} \dashv \Gamma[x \mapsto C \mathbin{@} s']}$$

Every method returns **Unit**. The environment update depends only on δ. Terminates (finite trees, each call strictly descends).

BICA correspondence. T-New $\leftrightarrow$ `initialStates.put(v, top())`; T-Call $\leftrightarrow$ `stateSpace.step(state, method)`.

4 λ_S^1: Method Return Values

Addition. Base types $\tau ::= \mathbf{Int} \mid \mathbf{Double} \mid \mathbf{String} \mid \mathbf{Bool}$. Method-signature table $msig(C, m) = \tau$. Constants c as values.

Modified rule.

$$\text{T-Call}^1 \ \frac{\Gamma(x) = C \mathbin{@} s \qquad \delta(s, m) = s' \qquad msig(C, m) = \tau}{\mathcal{CT};\ \Gamma \vdash x.m() : \tau \dashv \Gamma[x \mapsto C \mathbin{@} s']}$$

New rule.

$$\text{T-Const} \ \frac{typeof(c) = \tau}{\mathcal{CT};\ \Gamma \vdash c : \tau \dashv \Gamma}$$

Theorem 4.1 (Orthogonality of Returns)**.** *The output environment Γ' is independent of msig: changing return types does not affect session-state tracking.*

Proof. By structural induction. In every rule, $\Gamma \mapsto \Gamma'$ depends only on δ (from $\mathcal{CT}$), never on *msig*. □

Corollary 4.2. *A typestate checker that ignores return types entirely is sound with respect to λ_S^1.*

This is precisely what BICA Reborn's `TypestateChecker` does: it tracks `Map<String, Integer>` (variable $\rightarrow$ state ID) and never inspects Java return types.

5 λ_S^2: Conditionals and Selection

Phase transition (a): orthogonality breaks.

Addition. Selection types $\oplus\{l_1 : S_1, \ldots\}$. Enum types $\mathbf{Enum}(l_1, \ldots)$. Match expression.

Key rule.

$$\text{T-Match} \ \frac{\begin{array}{c} x : \mathbf{Enum}(l_1, \ldots, l_n) \\ \Gamma(y) = C \mathbin{@} s_\oplus \qquad \forall i.\ \delta_\oplus(s_\oplus, l_i) = s_i \qquad \forall i.\ \mathcal{CT};\ \Gamma[y \mapsto C \mathbin{@} s_i] \vdash e_i : T \dashv \Gamma' \end{array}}{\mathcal{CT};\ \Gamma \vdash \mathbf{match}\ x\ \mathbf{with}\ \{l_i \Rightarrow e_i\}_i : T \dashv \Gamma'}$$

Each branch is checked under a *different* successor state s_i. The enum return value determines which s_i the object enters: *orthogonality is broken.*

BICA correspondence. T-MATCH $\leftrightarrow$ `CallNode.SelectionSwitch` in `TypestateChecker.checkCallNodes`

6 λ_S^3: Recursion

Phase transition (b): termination breaks.

Addition. $S ::= \ldots \mid \mu X.\, S \mid X$. While expression: **while** x **do** e.

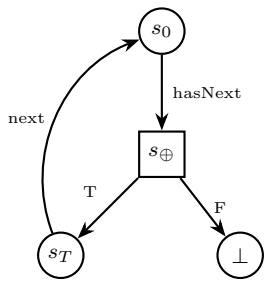
Key rule.

$$\text{T-WHILE}\ \frac{\Gamma(x) = C \mathbin{@} s \qquad s \neq \bot \qquad \mathcal{CT};\ \Gamma \vdash e : T \dashv \Gamma' \qquad \Gamma'(x) = C \mathbin{@} s' \qquad s' \in \mathrm{SCC}(s) \cup \{\bot\}}{\mathcal{CT};\ \Gamma \vdash \textbf{while}\ x\ \textbf{do}\ e : \textbf{Unit} \dashv \Gamma'[x \mapsto C \mathbin{@} \bot]}$$

The body leaves x in the same SCC (loop continues) or at $\bot$ (loop exits). Programs may diverge.

State spaces. Recursive types create cycles via placeholder-merge. The lattice check quotients by SCCs (Tarjan's algorithm).

Example: full Iterator. $S = \mu X.\, \&\{\mathit{hasNext} : \oplus\{\mathit{TRUE} : \&\{\mathit{next} : X\},\ \mathit{FALSE} : \textbf{end}\}\}$



7 λ_S^4: Parallel Composition

Phase transition (c): lattice structure emerges.

Addition. $S ::= \ldots \mid S_1 \parallel S_2$. Fork expression: **fork** $e_1 \parallel e_2$.

Product construction. $\mathcal{L}(S_1 \parallel S_2) = \mathcal{L}(S_1) \times \mathcal{L}(S_2)$, ordered componentwise. Top: $(\top_1, \top_2)$. Bottom: $(\bot_1, \bot_2)$.

Theorem 7.1 (Product Lattice)**.** *If $\mathcal{L}(S_1)$ and $\mathcal{L}(S_2)$ are lattices, then $\mathcal{L}(S_1) \times \mathcal{L}(S_2)$ is a lattice.*

This is why $\parallel$ *forces* lattice structure: concurrent access requires meets (synchronisation points) and joins (fork points), which exist precisely when the state space is a lattice.

Key rule.

$$\text{T-FORK}\ \frac{\begin{array}{c}\Gamma(x) = C \mathbin{@} (s_1, s_2) \\ \mathcal{CT};\ \Gamma[x \mapsto C \mathbin{@} (s_1, \top_2)] \vdash e_1 : T_1 \dashv \Gamma_1 \qquad \mathcal{CT};\ \Gamma[x \mapsto C \mathbin{@} (\top_1, s_2)] \vdash e_2 : T_2 \dashv \Gamma_2\end{array}}{\mathcal{CT};\ \Gamma \vdash \textbf{fork}\ e_1 \parallel e_2 : \textbf{Unit} \dashv \Gamma[x \mapsto C \mathbin{@} (\bot_1, \bot_2)]}$$

Theorem 7.2 (Thread Safety)**.** *Concurrent branches of* **fork** *access disjoint methods.*

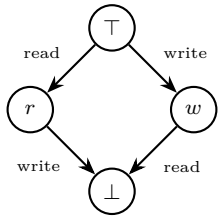
Table 2: Alignment test counts per calculus.

Calculus	Tests	Key test classes
λ_S^0	36	T-New, T-Call, T-Let, SessionCompletion, R-Call-Stuck
λ_S^1	21	T-Call-Ret, ConservativeExtension, Orthogonality
λ_S^2	13	SelectionStates, T-Match, OrthogonalityBreakdown
λ_S^3	11	CyclicStateSpaces, FullIterator, NonTermination
λ_S^4	21	ProductStateSpace, ConcurrentAccess, ThreadSafety
Total	**102**	

Example: concurrent read/write.

$$S = \&\{open : (\&\{read : \mathbf{end}\} \parallel \&\{write : \mathbf{end}\}) \,.\, \&\{close : \mathbf{end}\}\}$$

Product: $2 \times 2 = 4$-state diamond lattice.



8 Soundness Across the Hierarchy

Theorem 8.1 (Conservative Extension Chain)**.** *For each $i \in \{0, 1, 2, 3\}$, λ_S^{i+1} is a conservative extension of λ_S^i: every λ_S^i derivation is valid in λ_S^{i+1}.*

Theorem 8.2 (Preservation (λ_S^{0-4}))**.** *In every calculus of the hierarchy: if a well-typed configuration reduces, the result is well-typed.*

Theorem 8.3 (Progress (λ_S^{0-4}))**.** *In every calculus: a well-typed configuration is either a value or can reduce.*

Theorem 8.4 (Session Completion)**.** *(i) In λ_S^{0-2} (terminating): all programs complete sessions.*

(ii) In λ_S^{3-4} (non-terminating): if *a program terminates, all sessions are complete.*

9 Validation

9.1 Alignment Tests

Each typing rule has a corresponding test exercising BICA Reborn's `TypestateChecker`. Table 2 shows the test counts.

9.2 Lean 4 Mechanisation

Each calculus has four Lean files: Syntax, Typing, Semantics, Soundness (20 files total, ∼2,500 lines). Theorem statements and proof structures are mechanised; detailed proofs use `sorry` markers (tracked for completion).

9.3 Eclipse Test Projects

Five Eclipse projects (LambdaS0–LambdaS4) demonstrate the progression: from all-`void` methods (λ_S^0) through `String read()` (λ_S^1), enum switches (λ_S^2), `while` loops (λ_S^3), to concurrent fork/join (λ_S^4). All compile against BICA Reborn's annotation processor.

10 Related Work

Session types for channels. The VGR approach [10, 6] formalises session types over *channels* in a multithreaded functional language. Their typing judgement $\Gamma; \Sigma; e \mapsto \Sigma'; T; \Sigma''$ separates result types (T) from channel state (Σ), establishing the same orthogonality we prove in λ_S^1. The key difference: VGR works with binary channels (two endpoints with dual types), while λ_S works with objects (single entity, one protocol). Our $\|$ constructor is the object analogue of their channel-based concurrency.

Typestate. Strom and Yemini [9] introduced typestate. DeLine and Fähndrich [4] formalised typestates for objects with adoption and focus. Aldrich et al. [1] developed Plaid, a typestate-oriented language. These works predate the connection to session types and lattice structure.

Session types for objects. Gay et al. [5] introduced session types for distributed objects. Kouzapas et al. [8] developed Mungo, a Java front-end for session-type checking. Neither provides a stratified calculus hierarchy.

Featherweight calculi. Featherweight Java [7] pioneered the approach of stratifying language features into minimal calculi. λ_S follows this methodology for session-typed objects.

11 Conclusion

The λ_S hierarchy provides a formal foundation for session types on objects, stratified into five calculi that progressively add expressiveness. The three phase transitions—orthogonality breaking (λ_S^2), termination breaking (λ_S^3), and lattice emergence (λ_S^4)—identify the precise points where fundamental properties change.

The hierarchy connects practical implementation (BICA Reborn's `TypestateChecker`, validated by 102 tests) to algebraic theory (the Reticulate Theorem, mechanised in Lean 4). This bridge—from a working Java tool through formal calculi to mathematical structure—is the main contribution of the λ_S programme.

Future work. Session-typed method arguments (λ_S^5: ownership transfer) and session-typed return values (λ_S^6: factory patterns) require linearity or borrowing, connecting to recent work on borrowing for session types [3].

References

[1] Jonathan Aldrich, Joshua Sunshine, Darpan Saini, and Zachary Sparks. Typestate-oriented programming. In *OOPSLA 2009*, pages 1015–1022. ACM, 2009.

[2] Alexandre Zua Caldeira. Session types as algebraic reticulates: The lattice theorem (step 5), 2026. Submitted to ICE 2026.

[3] Luca Ciccone, Ornela Dardha, and Simon J. Gay. Borrowing from session types. *Proc. ACM Program. Lang.*, 9(OOPSLA), 2025.

[4] Robert DeLine and Manuel Fähndrich. Typestates for objects. In *ECOOP 2004*, volume 3086 of *LNCS*, pages 465–490. Springer, 2004.

[5] Simon Gay, Vasco T. Vasconcelos, António Ravara, Nils Gesbert, and Alexandre Zua Caldeira. Modular session types for distributed object-oriented programming. In *POPL 2010*, pages 299–312. ACM, 2010.

[6] Simon J. Gay, Nils Gesbert, António Ravara, and Vasco T. Vasconcelos. Modular session types for objects. *Logical Methods in Computer Science*, 11(4:12):1–76, 2015.

[7] Atsushi Igarashi, Benjamin C. Pierce, and Philip Wadler. Featherweight Java: A minimal core calculus for Java and GJ. *ACM Transactions on Programming Languages and Systems*, 23(3):396–450, 2001.

[8] Dimitrios Kouzapas, Ornela Dardha, Roly Perera, and Simon J. Gay. Typechecking protocols with Mungo and StMungo. In *PPDP 2016*, pages 146–159. ACM, 2016.

[9] Robert E. Strom and Shaula Yemini. Typestate: A programming language concept for enhancing software reliability. *IEEE Transactions on Software Engineering*, SE-12(1):157–171, 1986.

[10] Vasco T. Vasconcelos, Simon J. Gay, and António Ravara. Type checking a multithreaded functional language with session types. *Theoretical Computer Science*, 368(1–2):64–87, 2006.